1) **Análisis Experimental: Caída Intencional del Servidor**

Canal de Control (TCP)

Caída previa a la conexión (Servidor apagado)

- **Comportamiento observado:** Al iniciar ClienteTCP sin que ServidorTCP esté a la escucha en el puerto 5000 hace que la llamada al constructor new Socket (HOST, PUERTO) falle de forma inmediata.
  La excepción lanzada es: java.net.ConnectException: Connection refused: connect
- **Explicación:** TCP requiere establecer una conexión formal mediante el Three-Way Handshake . Al no existir un proceso enlazado (bind) al puerto TCP 5000, la pila del sistema operativo de destino responde con un paquete de control RST (Reset), notificando de inmediato al cliente que el puerto está cerrado.

Caída en caliente (Servidor terminado abruptamente durante la sesión)

- **Comportamiento observado:** Con el cliente conectado e interactuando, al finalizar forzosamente el proceso del servidor:
  1. Si el cliente estaba esperando respuesta tras enviar un comando (entrada.readLine()), la función retorna null o bien lanza una excepción si se intenta una nueva escritura.
  2. Si el cliente intenta enviar un nuevo comando mediante salida.println(lineaTeclado) y lee con entrada.readLine(), se captura:java.net.SocketException: Connection reset o en su defecto el cierre ordenado de fin de stream: "El servidor cerró la conexión."
- **Explicación técnica:** TCP es un protocolo orientado a conexión y con estado (*stateful*). Ambos extremos mantienen sincronizados contadores de secuencia y estados del socket. Cuando el proceso servidor muere, el sistema operativo envía un paquete FIN o responde con un RST ante cualquier paquete enviado a un socket huérfano. En Java, esto se traduce de inmediato en un EOF (null) o en una SocketException, garantizando que el cliente sepa al instante que el canal de control ya no es confiable.

1.2. Canal de Telemetría (UDP)

Caída del Receptor durante la transmisión

- **Comportamiento observado:** Al detener el proceso ReceptorUDP mientras EmisorUDP continúa generando lecturas cada 500 ms:

    - El emisor continúa ejecutando socket.send(paquete) en un bucle infinito sin arrojar ninguna excepción.
    - La consola del sensor sigue imprimiendo periódicamente:

  Enviado (#14): {"sensorId": 1, "temp": 24.12, "humedad": 55.30, "timestamp": 1729624500}
  Enviado (#15): {"sensorId": 1, "temp": 25.80, "humedad": 54.10, "timestamp": 1729624501}

- **Comportamiento silencioso:**
    - UDP es un protocolo no orientado a conexión y sin estado. No existe apretón de manos previo ni confirmación de entrega (*ACK*).
    - A nivel de API, DatagramSocket.send() simplemente copia el arreglo de bytes al buffer de salida del kernel local y retorna con éxito inmediatamente.
    - Aunque el sistema operativo destino pueda responder a nivel de red con un mensaje ICMP, la implementación estándar de datagramas no conectados en Java ignora y descarta silenciosamente estos avisos. El emisor asume que su trabajo terminó al colocar el paquete en el medio físico.

2. Cuadro Comparativo: TCP vs. UDP en la API de Java

Dimensión Técnica	TCP (java.net.Socket, ServerSocket)	UDP (java.net.DatagramSocket, DatagramPacket)
Sobrecarga de Conexión (Handshake)	**Alta (3-Way Handshake + 4-Way Teardown)** Requiere intercambio previo de paquetes SYN, SYN-ACK, ACK antes de transmitir el primer byte útil. Añade una latencia mínima de 1 RTT (Round-Trip Time).	**Nula (0 RTT)** No existe fase de conexión previa ni negociación de parámetros. El socket puede transmitir inmediatamente al instanciarse.
Orientación y Semántica	**Orientado a Flujo de Bytes (Stream)** Los datos se leen y escriben como un flujo continuo sin noción de límites de paquetes a nivel de capa de transporte.	**Orientado a Datagramas (Mensajes)** Cada paquete es una unidad discreta e independiente que preserva sus propios límites de mensaje (message boundaries).
Manejo de Flujo y Congestión	**Integrado y Adaptativo** Implementa control de flujo mediante ventana deslizante (Sliding Window) y control de congestión (Slow Start, Fast Recovery). Evita saturar al receptor o al canal.	**Inexistente** No modula la tasa de envío. Si el emisor inyecta paquetes a una frecuencia mayor a la capacidad de la red o del receptor, estos se descartan silenciosamente.
Comportamiento ante Bloqueo	**Bloqueo a nivel de flujo y Head-of-Line** accept() y readLine() bloquean el hilo hasta recibir datos o \n. Si se pierde un segmento, los posteriores quedan retenidos (Head-of-Line Blocking) hasta su retransmisión.	**Bloqueo aislado por datagrama** receive() bloquea hasta la llegada de un datagrama. No existe Head-of-Line Blocking; si un paquete se pierde, el siguiente se procesa de inmediato sin retrasar al receptor.
Gestión de Buffers en Java	**Abstracción de Streams (E/S de Alto Nivel)** No requiere reservar arreglos de bytes fijos en código (BufferedReader / PrintWriter). El tamaño de los buffers del socket (SO_RCVBUF, SO_SNDBUF) es administrado dinámicamente por la pila del SO.	**Buffers manuales fijos (byte[])** Requiere preasignar explícitamente un arreglo (ej. new byte[1024]). Si el paquete entrante excede el buffer, se trunca silenciosamente. Se debe controlar longitud y desplazamiento con getLength() y getOffset().
Detección de Caídas del Par	**Inmediata y Determinística** Se notifica a la aplicación mediante excepciones (ConnectException, SocketException) o valor de retorno null al alcanzar el fin de stream (EOF).	**Silenciosa (Fire-and-forget)** No hay mecanismo de aviso nativo. El emisor no sabe si el receptor está caído, fuera de línea, congestionado o procesando datos.